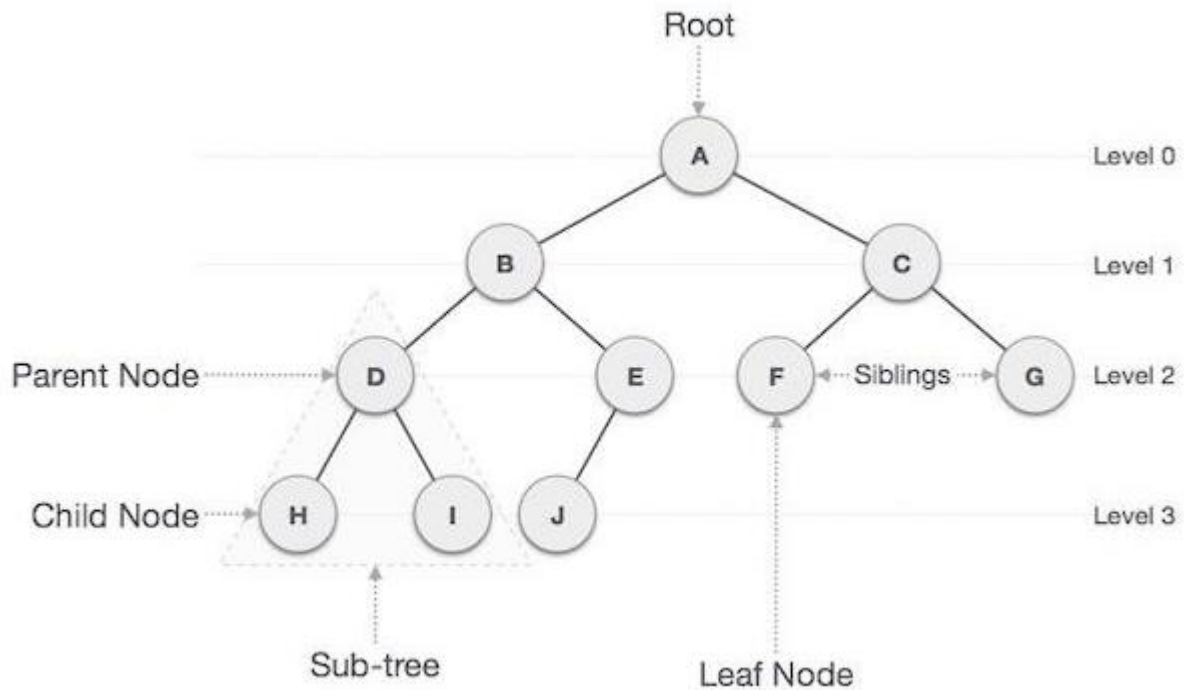


Tree:

So far, we have been studying mainly linear types of data structures: arrays, lists, stacks and queues. Now we define a nonlinear data structure called **Tree**. This structure is mainly used to represent data containing a hierarchical relationship between nodes/elements e.g. family trees and tables of contents.



A Tree is a non-linear abstract data type with a hierarchy-based structure. It consists of nodes (where the data is stored) that are connected via links. The tree data structure stems from a single node called a root node and has subtrees connected to the root.

Important Terms

The following are the important terms with respect to tree.

- **Path:** Path refers to the sequence of nodes along the edges of a tree.
- **Root:** The node at the top of the tree is called root. There is only one root per tree and one path from the root node to any node.
- **Parent:** Any node except the root node has one edge upward to a node called parent.
- **Child:** The node below a given node connected by its edge downward is called its child node.
- **Leaf:** The node which does not have any child node is called the leaf node.

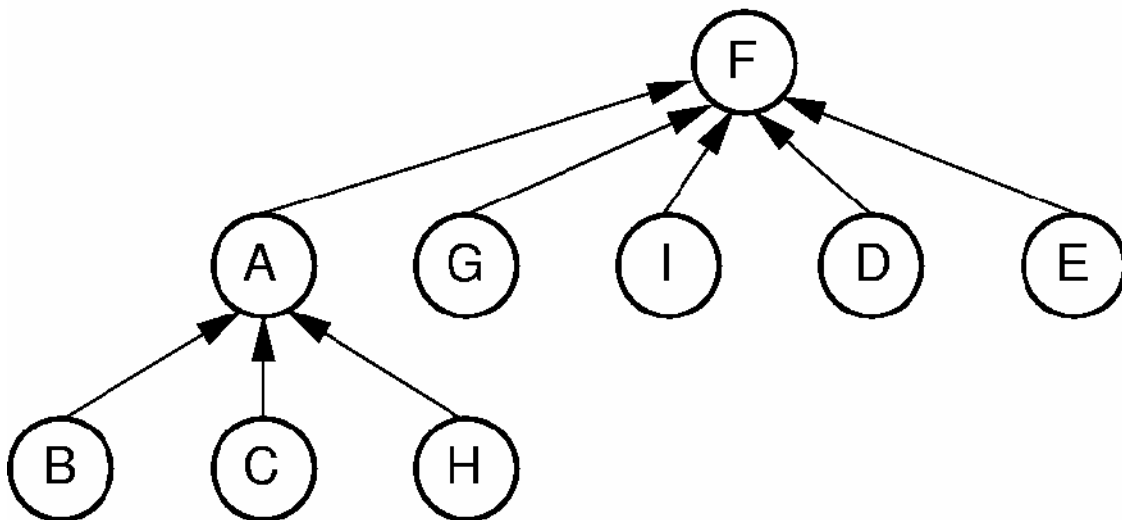
- **Subtree:** Subtree represents the descendants of a node.
- **Visiting:** Visiting refers to checking the value of a node when control is on the node.
- **Traversing:** Traversing means passing through nodes in a specific order.
- **Levels:** Level of a node represents the generation of a node. If the root node is at level 0, then its next child node is at level 1, its grandchild is at level 2, and so on.
- **Keys:** Key represents a value of a node based on which a search operation is to be carried out for a node.

There are two main types of tree:

- General Tree
- Binary Tree

General Tree:

A Tree where a node can have any number of children / descendants is called General Tree. For example:



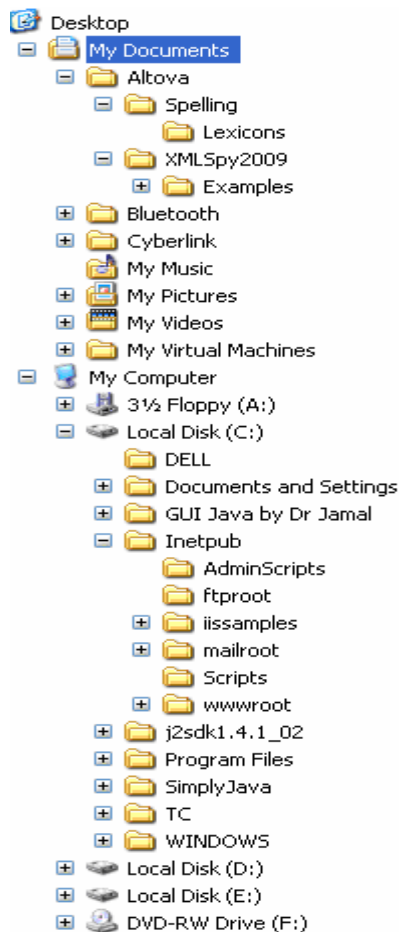
This following figure is a general tree where root is "Visual Programming".

Visual Programming

1 Introduction to the Visual Studio .NET IDE

1.1 Introduction	30
1.2 Visual Studio .NET Integrated Development Environment (IDE) Overview	34
1.3 Menu Bar and Toolbar	37
1.4 Visual Studio .NET Windows	39
1.4.1 Solution Explorer	39
1.4.2 Toolbox	40
1.4.3 Properties Window	42
1.5 Using Help	42
1.6 Simple Program: Displaying Text and an Image	44
2 ASP .NET, Web Forms and Web Controls	48
2.1 Introduction	49
2.2 Simple HTTP Transaction	50
2.3 System Architecture	52
2.4 Creating and Running a Simple Web Form Example	53
2.5 Web Controls	66
2.5.1 Text and Graphics Controls	67
2.5.2 AdRotator Control	71
2.5.2.1 X-axis Control	73
2.5.2.2 Y-axis Control	75
2.5.3 Validation Controls	76
2.6 Session Tracking	87
2.6.1 Cookies	88
2.6.2 Session Tracking with HttpSessionState	97
2.7 Case Study: Online Guest Book	100
2.8 Case Study: Connecting to a Database in ASP .NET	113

Following figure is also an example of general tree where root is "Desktop".

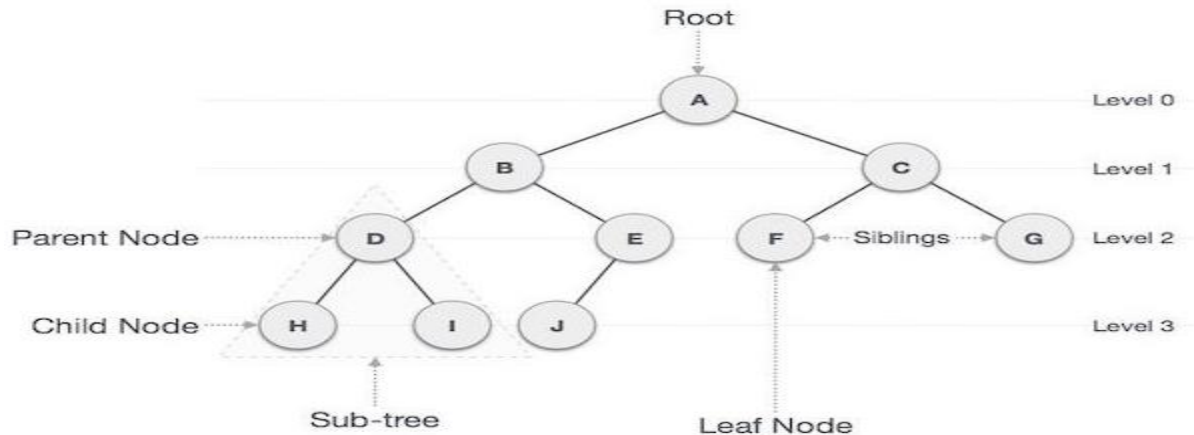


Binary Tree:

A tree in which each element may have 0-child, 1-child or maximum of 2-children.

A **Binary Tree T** is defined as a finite set of elements, called *nodes*, such that:

- T is empty (called the null tree or empty tree.)
- T contains a distinguished node **R** , called the *root* of T , and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2 .
-



If T does contain a *root* **R** , then the two trees T_1 and T_2 are called, respectively, the **left sub tree** and **right sub tree** of **R** .

If T_1 is non empty, then its node is called the left successor of **R** ; similarly, if T_2 is non empty, then its node is called the **right successor** of **R** .

The nodes with no successors are called the **terminal nodes**.

If N is a node in T with left successor S_1 and right successor S_2 , then N is called the **parent (or father)** of S_1 and S_2 . Analogously, S_1 is called the left child (or son) of N , and S_2 is called the right child (or son) of N . Furthermore, S_1 and S_2 are said to **siblings (or brothers)**.

Every node in the binary tree T , except the root, has a unique parent, called the predecessor of N .

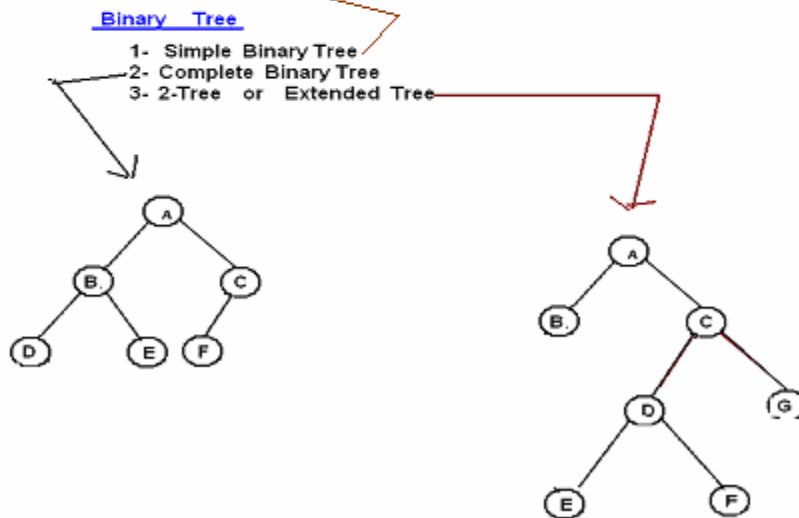
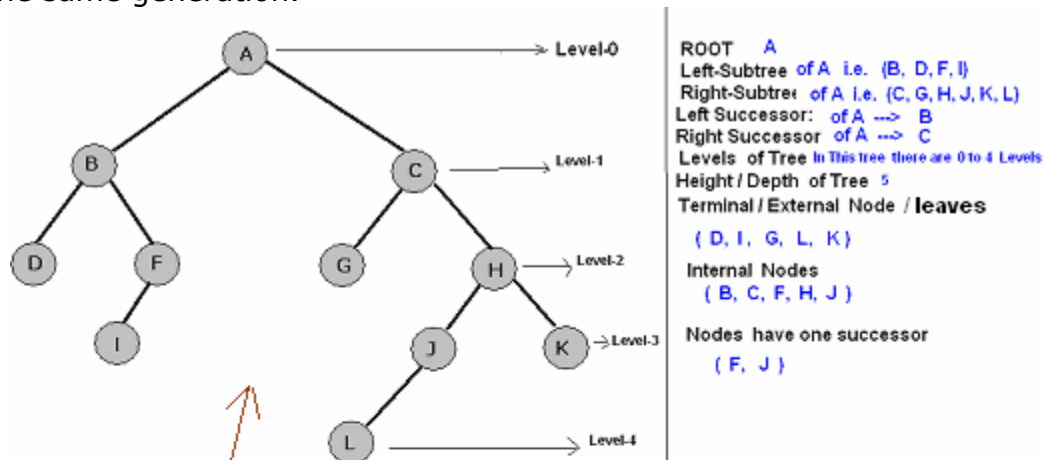
The line drawn from a node N of T to a successor is called an **edge**, and a sequence of consecutive edges is called a path.

A terminal node is called a **leaves**, and a path ending in a leaves is called a **branch**.

The **depth (or height)** of a tree T is the maximum number of nodes in a branch of T. This turns out to be 1 more than the largest level number of T.

Level of node & its generation:

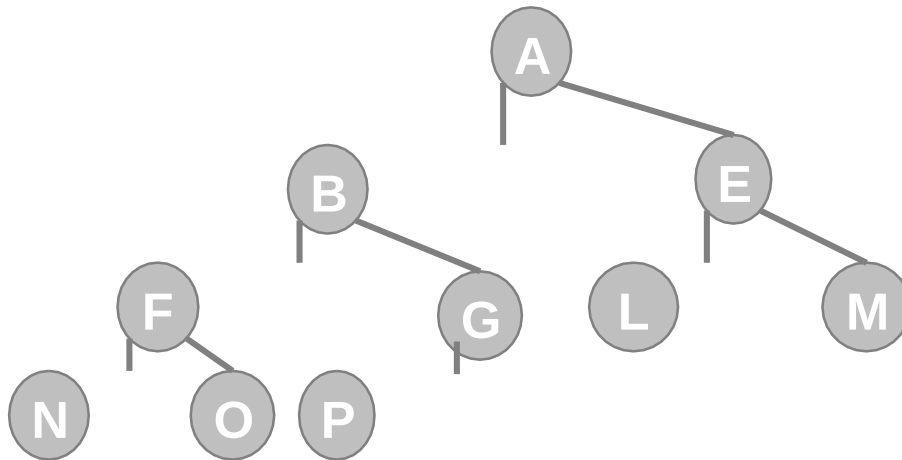
Each node in binary tree T is assigned a level number, as follows. The root R of the tree T is assigned the level number 0, and every other node is assigned a level number which is 1 more than the level number of its parent. Furthermore, those nodes with the same level number are said to belong to the same generation.



Complete Binary Tree:

Consider a binary tree T. each node of T can have at most two children. Accordingly, one can show that level **n-2** of T can have at most two nodes.

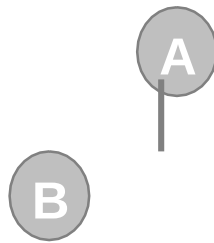
A Tree is said to be **complete** if all its levels, except possibly the last have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible.



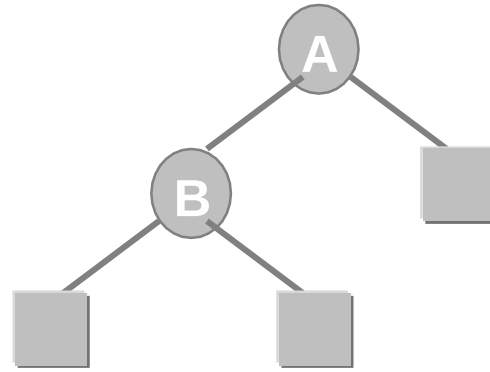
Extended Binary Tree: 2-Tree:

A binary tree T is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children. In such a case, the nodes, with 2 children are called internal nodes, and the nodes with 0 children are called external nodes.

- Extended binary tree consists of replacing every null subtree of the original tree with special nodes.
- Empty circle represents internal node and filled circle represents external node.
- The nodes from the original tree are internal nodes and the special nodes are external nodes.
- Every internal node in the extended binary tree has exactly two children and every external node is a leaf. It displays the result which is a complete binary tree.



Binary Tree



Extended 2-tree

Full Binary Tree

A full binary tree is a binary tree type where every node has either 0 or 2 child nodes.

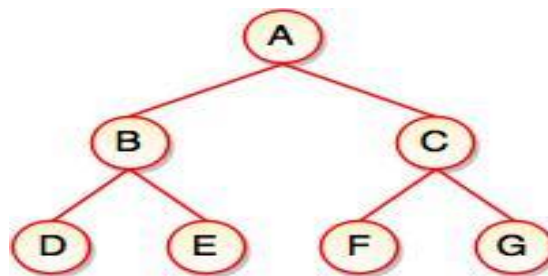


Fig. Full Binary Tree

Perfect Binary Tree

A perfect binary tree is a binary tree type where all the leaf nodes are on the same level and every node except leaf nodes have 2 children.

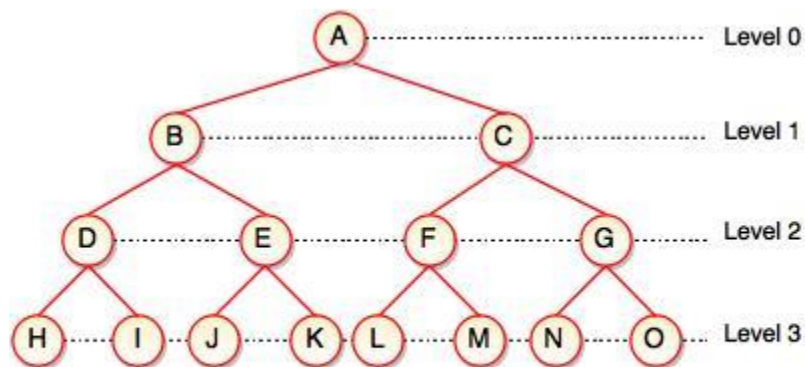


Fig. Complete Binary Tree

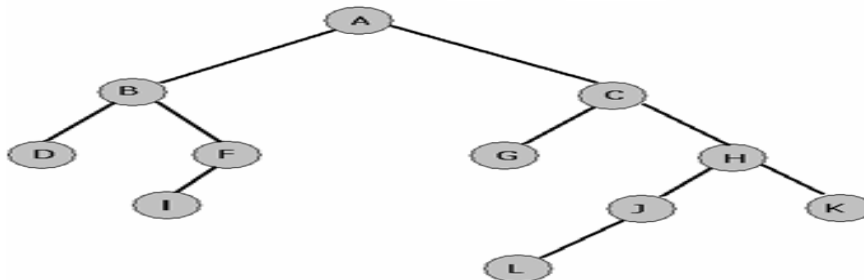
APPLICATIONS OF TREE DATA STRUCTURES

The following are the applications of trees:

- (a) **Storing naturally hierarchical data:** Trees are used to store the data in the hierarchical structure. For example, the file system. The file system stored on the disc drive, the file and folder are in the form of the naturally hierarchical data and stored in the form of trees.
- (b) **Organize data:** It is used to organize data for efficient insertion, deletion and searching. For example, a binary tree has a $\log N$ time for searching an element.
- (c) **Trie:** It is a special kind of tree that is used to store the dictionary. It is a fast and efficient way for dynamic spell checking.
- (d) **Heap:** It is also a tree data structure implemented using arrays. It is used to implement priority queues.
- (e) **B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.
- (f) **Routing table:** The tree data structure is also used to store the data in routing tables in the routers.

Traversing of Binary Tree:

A traversal of a tree T is a systematic way of accessing or visiting all the node of T. There are three standard ways of traversing a binary tree T with root R. these are:



The Tree diagram above will be used to explain the Three Traversal

- **Preorder (N L R):**

- a) Process the node/root. **(A B D F I C G H J L K)**
- b) Traverse the Left sub tree.
- c) Traverse the Right sub tree.

- **Inorder (L N R):** **(D B I F A G C L J H K)**
 - a) Traverse the Left sub tree.
 - b) Process the node/root.
 - c) Traverse the Right sub tree.

- **Postorder (L R N):** **(D I F B G L J K H C A)**
 - a) Traverse the Left sub tree.
 - b) Traverse the Right sub tree.
 - c) Process the node/root.

- **Descending order (R N L):** **(K H J L C G A F I B D)**
(Used in Binary Search Tree, will be discussed later)
 - a) Traverse the Right sub tree.
 - b) Process the node/root.
 - c) Traverse the Left sub tree.

Preorder Traversal:

Algorithm: PREORDER (pRoot)

First time this function is called by passing original **root** into **pRoot**. Here **pRoot** is pointers pointing to current root. This algorithm does a preorder traversal of T, applying by rcurrensively calling same function and updating **pRoot** to traverse in a way i.e. **(NLR)** Node, Left, Right.

1. If (pRoot NOT = NULL) then: [does child exist ?]
 - Apply PROCESS to pRoot-> info. e.g. Write: pRoot -> info
 - [recursive call by passing address of left child to update **pRoot**]
 - i PREORDER (pRoot -> Left)
 - [recursive call by passing address of right child to update **pRoot**]
 - ii PREORDER(pRoot -> Right)
 - [End of If structure.]

2. Exit

Inorder Traversal:

Algorithm: INORDER (pRoot)

First time this function is called by passing original **root** into **pRoot**. Here **pRoot** is pointers pointing to current root. This algorithm does a Inorder traversal of T, applying by rcursively calling same function and updating **proot** to traverse in a way i.e. (**LNR**) Left, Node, Right.

1. If (pRoot NOT = NULL) then: [does child exist ?]
 [recursive call by passing address of left child to update **pRoot**]
 i- PREORDER (pRoot -> Left)
 ii- Apply PROCESS to pRoot-> info. e.g. Write: pRoot -> info
 [recursive call by passing address of right child to update **pRoot**]
 iii- PREORDER(pRoot -> Right)
 [End of If structure.]
2. Exit.

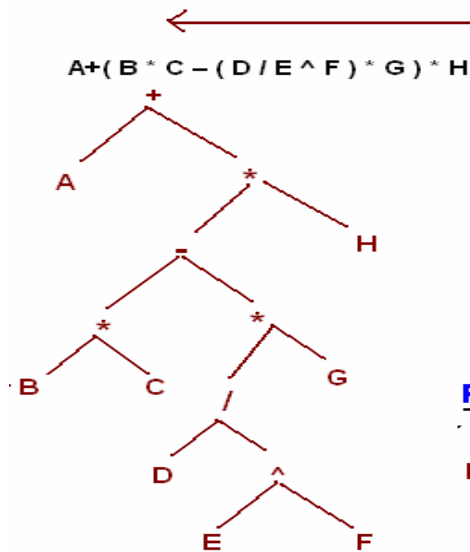
Postorder Traversal:

Algorithm: POSTORDER (pRoot)

First time this function is called by passing original **root** into **pRoot**. Here **pRoot** is pointers pointing to current root. This algorithm does a Postorder traversal of T, applying by rcursively calling same function and updating **proot** to traverse in a way i.e. (**LRN**) Left, Right, Node.

1. If (pRoot NOT = NULL) then: [does child exist ?]
 [recursive call by passing address of left child to update **pRoot**]
 i- PREORDER (pRoot -> Left)
 [recursive call by passing address of right child to update **pRoot**]
 ii- PREORDER(pRoot -> Right)
 iii- Apply PROCESS to pRoot-> info. e.g. Write: pRoot -> info
 [End of If structure.]
2. Exit.

Preparing a Tree from an infix arithmetic expression



Find operator which has low priority with respect to execution, starting from right to left. Put it on root and then continue this processor on sub-trees. The infix expression on the left side is converted into tree. Examine this tree is a 2-Tree, where each node either has two nodes or zero nodes.

All internal nodes are Operators

+ * - * * /

and all leaf nodes are Operands

A H B C G D E F

Post Order Traversing (LRN)

ABC * DEF ^ / G * - H * +

It produce postfix expression

Pre-Order Traversing (NLR)

+ A * - * B C * / D ^ E F G H

it produce prefix arithmetic expression

Recursion

For implementing tree traversal logics as stated, two approaches are used, i.e. use stacks or recursive functions. In this lecture notes, we will see only recursion approach to implement these algorithms. Students will also use stacks to implement these algorithms as homework.

Recursion is an important concept in computer science. Many algorithms can be best described in terms of recursion. A recursive function / procedure containing a Call statement to itself. To make a function recursive one must consider the following properties:

- (1) There must be certain (using arguments), called **base criteria**, for which the procedure / function does not call itself.
- (2) Each time the procedure / function does call itself, control must be closer to the base criteria.

Following examples helps to clarify of these ideas:

/* The following program demonstrates function call of itself. The following function does not follow the first property i.e. (recursion must have some criteria). Endless execution */

```
#include <stdio.h>

void disp( ); // prototyping
int main( )
{
    printf("\nHello");
    disp( ); /* A function, call */
    return 0;
}

void disp( )
{ printf("Hello\t");
  disp( ); /* A function call to itself without any criteria */
}
```

Don't run above program, it is still an explanation thus program is not valid logically.

The second property of recursion i.e. (after each cycle/iteration control must reach closer to the base criteria) and it is ignored here, so the following program is logically invalid.

```
#include <stdio.h>
void numbers(int );
int main( )
{
    int i=10;
    numbers(n);
    return 0;
}

void numbers(int n )
{
    printf("Hello\t");
    if(n >= 1 ) numbers(n+1); // this needs explanation
    // after each iteration, control is going far from base criteria
}
```

Factorial Function:

The product of the positive integers from ***n* to 1**, inclusive, is called "***n*_factorial**" and is usually denoted by ***n*!**.

It is also convenient to define $0! = 1$, so that the function is defined for all nonnegative integers. Thus we have:

$$\begin{aligned}
 0! &= 1 & 1! &= 1 & 2! &\rightarrow 2*1 = 2 & 3! &\rightarrow 3*2*1 = 6 \\
 4! &\rightarrow 4*3*2*1 = 24 & 5! &\rightarrow 5*4*3*2*1 = 120 & 6! &\rightarrow 6*5*4*3*2*1 = 720
 \end{aligned}$$

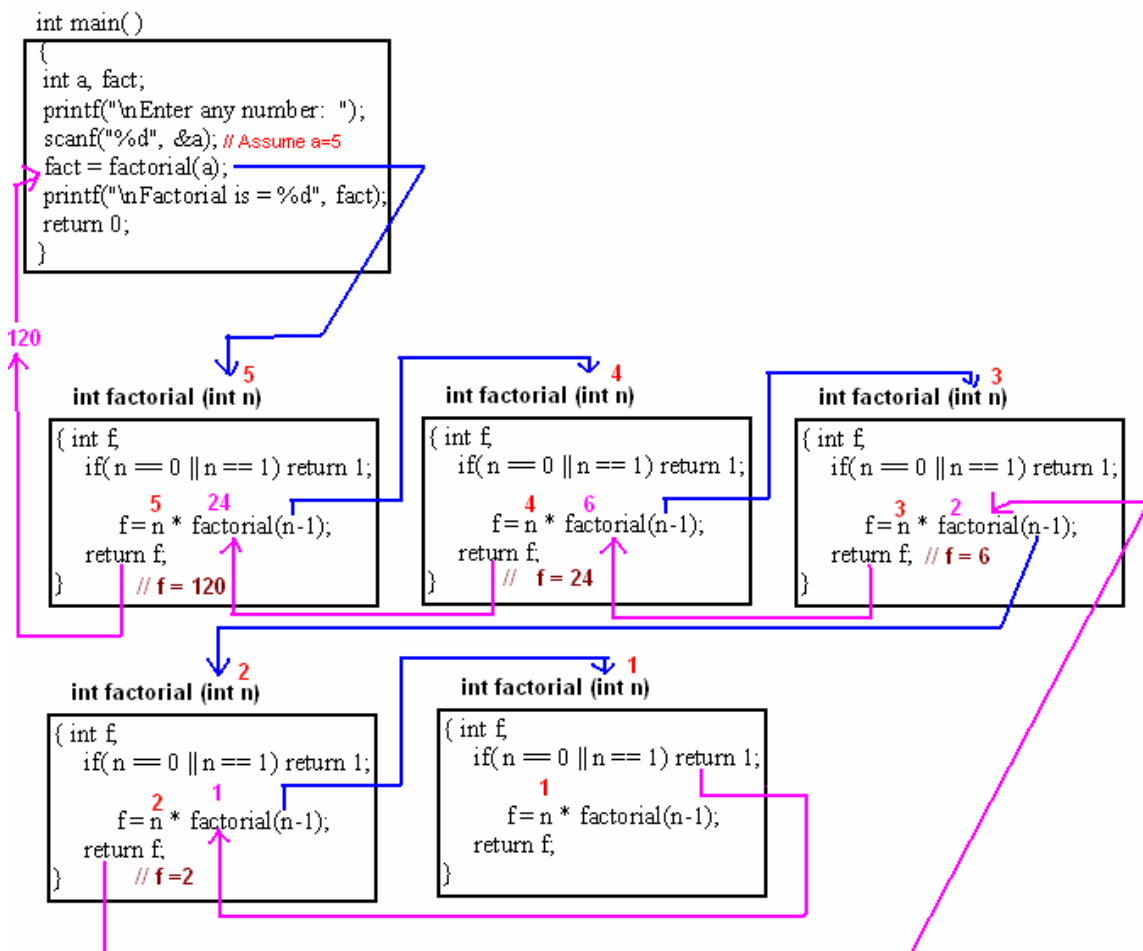
Program to find the factorial of the given number using for loop:

To find the factorial of a given number using recursion

DATA STRUCTURES (CSC-214)

```
#include <stdio.h>
int factorial(int);
void main
{ int a, fact;
  printf("\nEnter any number: ");
  scanf("%d", &a);
  fact = factorial(a);
  printf("\nFactorial is = %d", fact);
}
int factorial(int n)
{   int f = 1, i;
    for( i = n; i>=1; i--)
        f = f * i;
    return f;
}
```

```
#include <stdio.h>
int factorial(int);
int main( )
{ int a, fact;
  printf("\nEnter any number: ");
  scanf("%d", &a);
  fact = factorial(a);
  printf("\nFactorial is = %d", fact);
  return 0;
}
int factorial(int n)
{ int f;
  if( n == 0 || n == 1) return 1;
  f = n * factorial(n-1);
  return f;
}
```

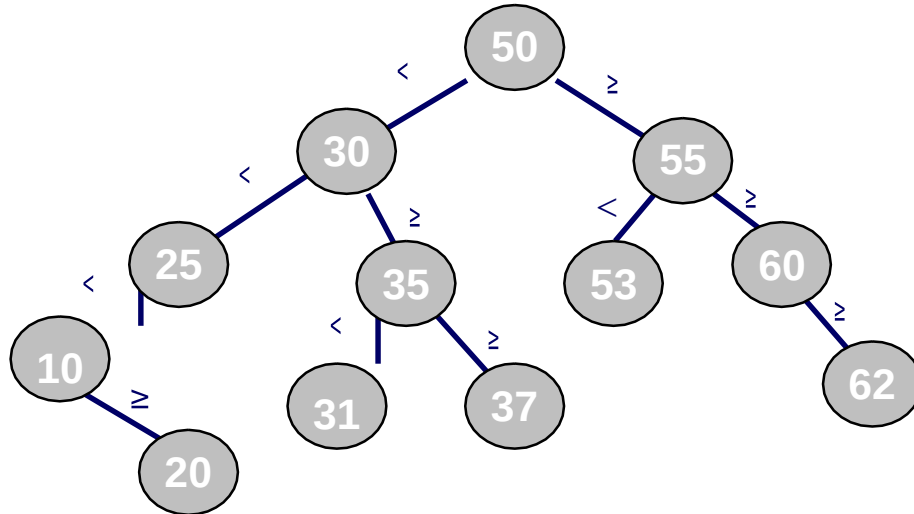


Binary Search Tree:

Suppose T is a binary tree, the T is called a binary search tree or binary sorted tree if each node N of T has the following property:

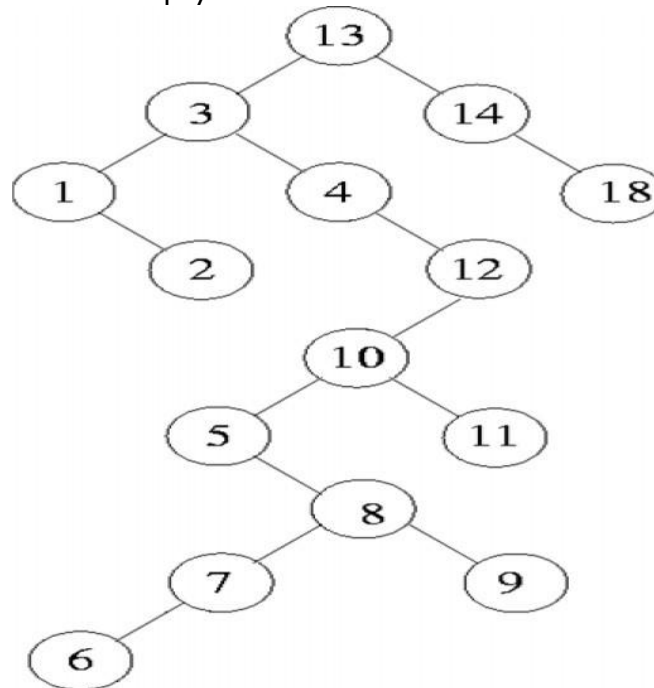
The values of at N (node) is greater than every value in the left sub tree of N and is less than every value in the right sub tree of N.

Binary Search Tree using these values: (50, 30, 55, 25, 10, 35, 31, 20, 53, 60,



62)

Following figure shows a binary search tree. Notice that this tree is obtained by inserting the values 13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18 in that order, starting from an empty tree.



- **Sorting:** Note that inorder traversal of a binary search tree always gives a sorted sequence of the values. This is a direct consequence of the BST property. This provides a way of sorting a given sequence of keys: first, create a BST with these keys and then do an inorder traversal of the BST so created.

Inorder Travers (LNR) : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 18

- **Search:** is straightforward in a BST. Start with the root and keep moving left or right using the BST property. If the key we are seeking is present, this search procedure will lead us to the key. If the key is not present, we end up in a null link.
- **Insertion** in a BST is also a straightforward operation. If we need to insert an element **n**, we traverse tree starting from root by considering the above stated rules. Our traverse procedure ends in a null link. It is at this position of this null link that **n** will be included. .
- **Deletion in BST:** Let x be a value to be deleted from the BST and let N denote the node containing the value x. Deletion of an element in a BST again uses the BST property in a critical way. When we delete the node N containing x, it would create a "gap" that should be filled by a suitable existing node of the BST. There are two possible candidate nodes that can fill this gap, in a way that the BST property is not violated: (1). Node containing highest valued element among all descendants of left child of N. (2). Node containing the lowest valued element among all the descendants of the right child of N. There are three possible cases to consider:

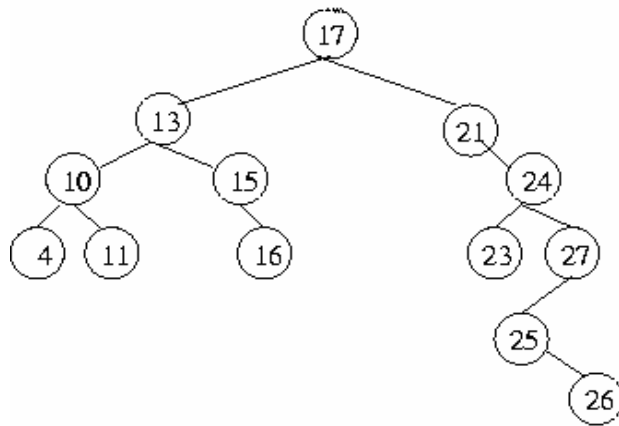
Deleting a leaf (node with no children): Deleting a leaf is easy, as we can simply remove it from the tree.

Deleting a node with one child: Delete it and replace it with its child.

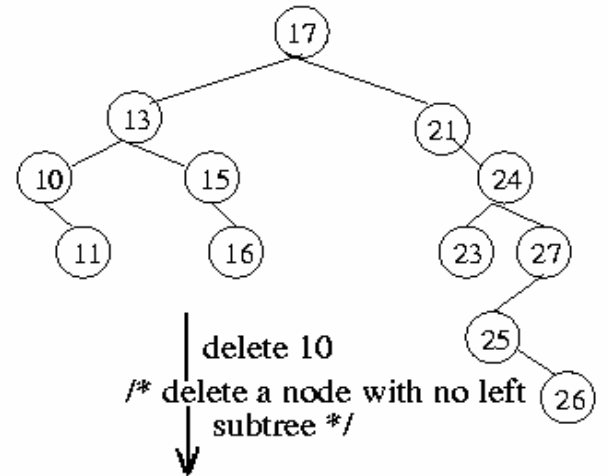
Deleting a node with two children: Call the node to be deleted "N". Do not delete N. Instead, choose its in-order successor node "S". Replace the value of "N" with the value of "S". (Note: S itself has up to one child.)

As with all binary trees, a node's in-order successor is the left-most child of its right subtree. This node will have zero or one child. Delete it according to one of the two simpler cases above.

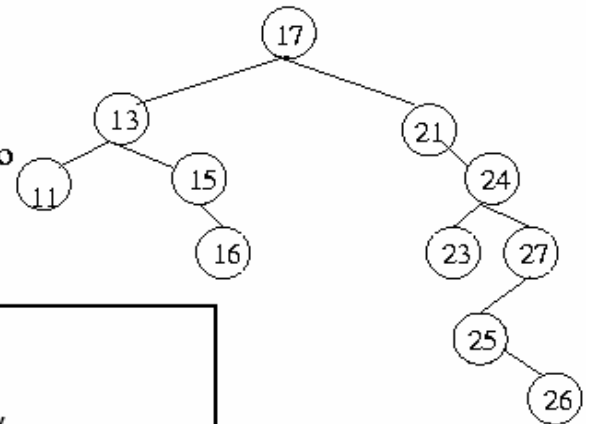
Figure on next page illustrates several scenarios for deletion in BSTs.



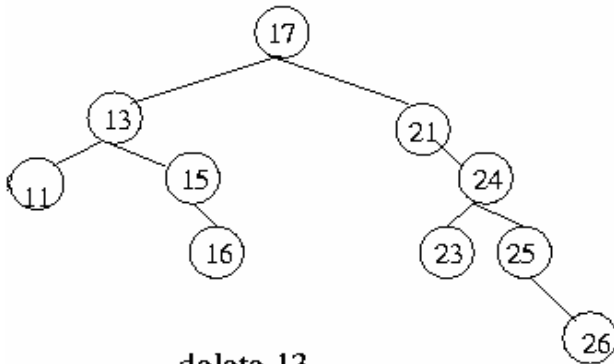
delete 4
/* delete leaf */



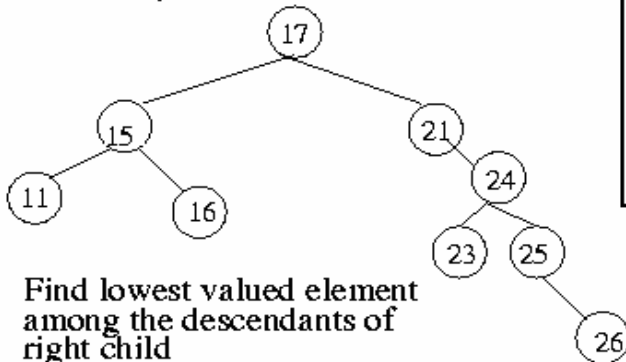
delete 10
/* delete a node with no left subtree */



delete 27
/* delete node with no right subtree */



delete 13
/* delete node with both left and right subtrees */



Find lowest valued element among the descendants of right child

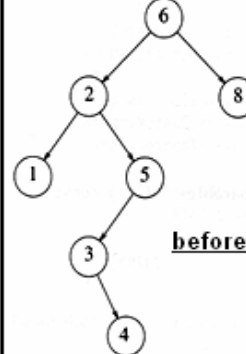
OR Find the in-order Successor and replace it with node to be deleted

delete

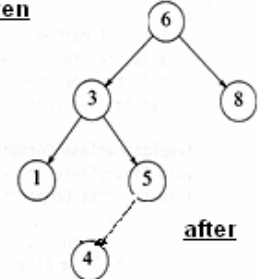
Three cases:

- [1] the node is a leaf
Delete it immediately
- [2] the node has one child
Adjust a pointer from the parent to bypass that node
- [3] the node has 2 children
replace the value of that node with the minimum element at the right subtree.
delete the minimum element Has either no child or one child.
So invoke case 1 or 2

Second example to delete node has two children



before



after

Deletion of a node (2) with two children, before and after